tinytable

Easy, beautiful, and customizable tables in R

Table of contents

1 Tiny Tables **3**
- 1.1 Output formats . . . 4
- 1.2 Themes . . . 6
- 1.3 Alignment . . . 6
- 1.4 Formatting (numbers, dates, strings, etc.) . . . 7
- 1.5 Width . . . 9
- 1.6 Line breaks and text wrapping . . . 10
- 1.7 Captions and cross-references . . . 11
- 1.8 Footnotes . . . 12
- 1.9 Math . . . 13
- 1.10 Markdown . . . 13

2 Style **15**
- 2.1 Cells, rows, columns . . . 16
- 2.2 Colors . . . 19
- 2.3 Fonts . . . 20
- 2.4 Spanning cells . . . 20
- 2.5 Headers . . . 21
- 2.6 Conditional styling . . . 22
- 2.7 Vectorized styling (heatmaps) . . . 23
- 2.8 Lines (borders) . . . 24

3 Tiny plots and images **26**
- 3.1 Inserting images in tables . . . 26
- 3.2 Inline plots . . . 26
  - 3.2.1 Built-in plots . . . 27
  - 3.2.2 Custom plots: Base `R` . . . 27
  - 3.2.3 Custom plots: `ggplot2` . . . 28

4 Groups and labels **30**
4.1 Rows 30
4.2 Columns 32

5 HTML customization **34**
5.1 Themes 34
5.2 CSS declarations 34
5.3 CSS rules 34

6 LaTeX / PDF customization **34**
6.1 Preamble 34
6.2 Introduction to `tabularray` 35
6.3 `tabularray` keys 37

`tinytable` is a small but powerful `R` package to draw HTML, LaTeX, Word, PDF, Markdown, and Typst tables. The interface is minimalist, but it gives users direct and convenient access to powerful frameworks to create endlessly customizable tables.

Install it from Github:

```
library(remotes)
install_github("vincentarelbundock/tinytable")
```

This tutorial introduces the main functions of the package. It is available in two versions:

- PDF
- HTML

1 Tiny Tables

Load the library and set some global options:

```
library(tinytable)

options(digits = 3) # how many significant digits to print by default
options("tinytable_tabularray_placement" = "H") # for LaTeX
```

Draw a first table:

```
x <- mtcars[1:4, 1:5]
tt(x)
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

1.1 Output formats

`tinytable` can produce tables in HTML, Word, Markdown, LaTeX, PDF, or PNG format. An appropriate output format for printing is automatically selected based on (1) whether the function is called interactively, (2) is called within RStudio, and (3) the output format of the Rmarkdown or Quarto document, if applicable. Alternatively, users can specify the print format in `print()` or by setting a global option:

```
tt(x) |> print("markdown")
tt(x) |> print("html")
tt(x) |> print("latex")

options(tinytable_print_output = "markdown")
```

With the `save_tt()` function, users can also save tables directly to PNG (images), PDF or Word documents, and to any of the basic formats. All we need to do is supply a valid file name with the appropriate extension (ex: `.png`, `.html`, `.pdf`, etc.):

```
tt(x) |> save_tt("path/to/file.png")
tt(x) |> save_tt("path/to/file.pdf")
tt(x) |> save_tt("path/to/file.docx")
tt(x) |> save_tt("path/to/file.html")
tt(x) |> save_tt("path/to/file.tex")
tt(x) |> save_tt("path/to/file.md")
```

`save_tt()` can also return a string with the table in it, for further processing in `R`. In the first case, the table is printed to console with `cat()`. In the second case, it returns as a single string as an `R` object.

```
tt(mtcars[1:10, 1:5]) |>
  group_tt(
    i = list(
      "Hello" = 3,
      "World" = 8),
    j = list(
      "Foo" = 2:3,
      "Bar" = 4:5)) |>
  print("markdown")
```

```
+------+-------------+------------+
```

```
|      | Foo         | Bar        |
+------+-----+-------+-----+------+
| mpg  | cyl | disp  | hp  | drat |
+======+=====+=======+=====+======+
| 21   | 6   | 160   | 110 | 3.9  |
+------+-----+-------+-----+------+
| 21   | 6   | 160   | 110 | 3.9  |
+------+-----+-------+-----+------+
| Hello                           |
+------+-----+-------+-----+------+
| 22.8 | 4   | 108   | 93  | 3.85 |
+------+-----+-------+-----+------+
| 21.4 | 6   | 258   | 110 | 3.08 |
+------+-----+-------+-----+------+
| 18.7 | 8   | 360   | 175 | 3.15 |
+------+-----+-------+-----+------+
| 18.1 | 6   | 225   | 105 | 2.76 |
+------+-----+-------+-----+------+
| 14.3 | 8   | 360   | 245 | 3.21 |
+------+-----+-------+-----+------+
| World                           |
+------+-----+-------+-----+------+
| 24.4 | 4   | 146.7 | 62  | 3.69 |
+------+-----+-------+-----+------+
| 22.8 | 4   | 140.8 | 95  | 3.92 |
+------+-----+-------+-----+------+
```

```r
tt(mtcars[1:10, 1:5]) |>
  group_tt(
    i = list(
      "Hello" = 3,
      "World" = 8),
    j = list(
      "Foo" = 2:3,
      "Bar" = 4:5)) |>
  save_tt("markdown")
```

```
[1] "+------+-------------+------------+\n|      | Foo         | Bar        |\n+------+------
```

1.2 Themes

`tinytable` offers a few basic themes out of the box: "default", "striped", "grid", "void." Those themes can be applied with the `theme` argument of the `tt()` function. As we will see below, it is easy to go much beyond those basic settings to customize your own tables. Here we only illustrate a few of the simplest settings:

```
tt(x, theme = "striped")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

```
tt(x, theme = "grid")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

```
tt(x, theme = "void")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

1.3 Alignment

To align columns, we use a single character, or a string where each letter represents a column:

```r
dat <- data.frame(
  a = c("a", "aa", "aaa"),
  b = c("b", "bb", "bbb"),
  c = c("c", "cc", "ccc"))

tt(dat) |> style_tt(j = 1:3, align = "c")
```

a	b	c
a	b	c
aa	bb	cc
aaa	bbb	ccc

```r
tt(dat) |> style_tt(j = 1:3, align = "lcr")
```

a	b	c
a	b	c
aa	bb	cc
aaa	bbb	ccc

1.4 Formatting (numbers, dates, strings, etc.)

The `tt()` function is minimalist; it's inteded purpose is simply to draw nice tables. Users who want to format numbers, dates, strings, and other variables in different ways should process their data *before* supplying it to the `tt()` table-drawing function. To do so, we can use the `format_tt()` function supplied by the `tinytable`.

In a very simple case—such as printing 2 significant digits of all numeric variables—we can use the `digits` argument of `tt()`:

```r
dat <- data.frame(
     w = c(143002.2092, 201399.181, 100188.3883),
     x = c(1.43402, 201.399, 0.134588),
     y = as.Date(sample(1:1000, 3)),
     z = c(TRUE, TRUE, FALSE))

tt(dat, digits = 2)
```

w	x	y	z
143002	1.43	1970-05-25	True
201399	201.40	1970-02-24	True
100188	0.13	1971-09-24	False

We can get more fine-grained control over formatting by calling `format_tt()` after `tt()`, optionally by specifying the columns to format with `j`:

```
tt(dat) |>
  format_tt(
    j = 2:4,
    digits = 1,
    date = "%B %d %Y") |>
  format_tt(
    j = 1,
    digits = 2,
    num_mark_big = " ",
    num_mark_dec = ",",
    num_fmt = "decimal")
```

w	x	y	z
143 002,21	1.4	May 25 1970	True
201 399,18	201.4	February 24 1970	True
100 188,39	0.1	September 24 1971	False

We can use a regular expression in `j` to select columns, and the `?sprintf` function to format strings, numbers, and to do string interpolation (similar to the `glue` package, but using Base `R`):

```
dat <- data.frame(
     a = c("Burger", "Halloumi", "Tofu", "Beans"),
     b = c(1.43202, 201.399, 0.146188, 0.0031),
     c = c(98938272783457, 7288839482, 29111727, 93945))
tt(dat) |>
  format_tt(j = "a", sprintf = "Food: %s") |>
  format_tt(j = 2, digits = 1) |>
  format_tt(j = "c", digits = 2, num_suffix = TRUE)
```

a	b	c
Food: Burger	1.432	98.94T
Food: Halloumi	201.399	7.29B
Food: Tofu	0.146	29.11M
Food: Beans	0.003	93.94K

Finally, if you like the `format_tt()` interface, you can use it directly with numbers, vectors, or data frames:

```
format_tt(pi, digits = 1)
```

```
[1] "3"
```

```
format_tt(dat, digits = 1, num_suffix = TRUE)
```

```
         a     b     c
1   Burger   1.4 98.9T
2 Halloumi 201.4  7.3B
3     Tofu   0.1 29.1M
4    Beans   0.0 93.9K
```

1.5 Width

The `width` arguments accepts a number between 0 and 1, indicating what proportion of the linewidth the table should cover:

```
tt(x, width = 0.5)
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

```r
tt(x, width = 1)
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

1.6 Line breaks and text wrapping

When the `width` argument is specified and a cell includes long text, the text is automatically wrapped to match the table.

```r
lorem <- data.frame(
  Lorem = "Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium dolorem
  Ipsum = " Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed q
)
tt(lorem, width = 3/4)
```

Table 1: A full width table with wrapped text.

Lorem	Ipsum
Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo.	Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos.

Manual line breaks work sligthly different in LaTeX (PDF) or HTML. This table shows the two strategies. For HTML, we insert a `<br>` tag. For LaTeX, we wrap the string in curly braces `{}`, and then insert two (escaped) backslashes: `\\\\`

```r
d <- data.frame(
  "{Sed ut \\\\ perspiciatis unde}",
  "dicta sunt<br> explicabo. Nemo"
```

Table 2: Blah blah blah

mpg	cyl	disp	hp
21	6	160	110
21	6	160	110
22.8	4	108	93
21.4	6	258	110

```
) |> setNames(c("LaTeX line break", "HTML line break"))
tt(d, width = 1)
```

LaTeX line break	HTML line break
Sed ut perspiciatis unde	dicta sunt explicabo. Nemo

1.7 Captions and cross-references

In Quarto, one can specify captions and use cross-references using code like this:

```
@tbl-blah shows that...

```{r}
#| label: tbl-blah
#| tbl-cap: "Blah blah blah"
library(tinytable)
tt(mtcars[1:4, 1:4])
```
```

And here is the rendered version of the code chunk above:

Table 2 shows that…

```
library(tinytable)
tt(mtcars[1:4, 1:4], placement = NULL)
```

For standalone LaTeX tables, you can use the `caption` argument like so:

```
tt(x, caption = "Blah blah.\\label{tbl-blah}")
```

Be aware that this more approach may not work well in Quarto or Rmarkdown documents.

1.8 Footnotes

The `notes` argument accepts single strings or named lists of strings:

```
n <- "Fusce id ipsum consequat ante pellentesque iaculis eu a ipsum. Mauris id ex in nulla co

tt(lorem, notes = n, width = 1)
```

Table 3: A full-width table with wrapped text in cells and a footnote.

Lorem	Ipsum
Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo.	Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos.

Fusce id ipsum consequat ante pellentesque iaculis eu a ipsum. Mauris id ex in nulla consectetur aliquam. In nec tempus diam. Aliquam arcu nibh, dapibus id ex vestibulum, feugiat consequat erat. Morbi feugiat dapibus malesuada. Quisque vel ullamcorper felis. Aenean a sem at nisi tempor pretium sit amet quis lacus.

When `notes` is a named list, the names are used as identifiers and displayed as superscripts:

```
tt(x, notes = list(a = "Blah.", b = "Blah blah."))
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

[a] Blah.
[b] Blah blah.

1.9 Math

To insert LaTeX-style mathematical expressions in a `tinytable`, we enclose the expression in dollar signs: `$...$`. The expression will then rendered as a mathematical expression by MathJax (for HTML), LaTeX, or Pandoc. Do not forget to double escape any backslashes.

```
dat <- data.frame(Math = c(
  "$x^2 + y^2 = z^2$",
  "$\\frac{1}{2}$"
))
tt(dat) |> style_tt(j = 1, align = "c")
```

Math
$x^2 + y^2 = z^2$
$\frac{1}{2}$

In LaTeX (PDF), you can also use the `mode` inner setting from `tabularray` to render math in tables without delimiters (see Section 6 for details on `tabularray`):

```
dat <- data.frame(Math = c("x^2 + y^2 = z^2", "\\frac{1}{2}"))
tt(dat) |>
  style_tt(j = 1, align = "c", tabularray_inner = "column{1}={mode=math},")
```

$Math$
$x^2 + y^2 = z^2$
$\frac{1}{2}$

1.10 Markdown

Markdown can be rendered in cells by using the `markdown` argument of the `format_tt()` function (note: this requires installing the `markdown` as an optional dependency).

```
dat <- data.frame( markdown = c(
  "This is _italic_ text.",
  "This sentence ends with a superscript.^2^")
)
```

```r
tt(dat) |>
  format_tt(j = 1, markdown = TRUE) |>
  style_tt(j = 1, align = "c")
```

markdown
This is *italic* text.
This sentence ends with a superscript.[2]

Markdown syntax can be particularly useful when formatting URL links in a table:

```r
dat <- data.frame(
  `Package (link)` = c(
    "[`marginaleffects`](https://www.marginaleffects.com/)",
    "[`modelsummary`](https://www.modelsummary.com/)",
    "[`tinytable`](https://vincentarelbundock.github.io/tinytable/)",
    "[`countrycode`](https://vincentarelbundock.github.io/countrycode/)",
    "[`WDI`](https://vincentarelbundock.github.io/WDI/)",
    "[`softbib`](https://vincentarelbundock.github.io/softbib/)",
    "[`tinysnapshot`](https://vincentarelbundock.github.io/tinysnapshot/)",
    "[`altdoc`](https://etiennebacher.github.io/altdoc/)",
    "[`plot2`](https://grantmcdermott.com/plot2/)",
    "[`parameters`](https://easystats.github.io/parameters/)",
    "[`insight`](https://easystats.github.io/insight/)"
  ),
  Purpose = c(
    "Interpreting statistical models",
    "Data and model summaries",
    "Draw beautiful tables easily",
    "Convert country codes and names",
    "Download data from the World Bank",
    "Software bibliographies in R",
    "Snapshots for unit tests using `tinytest`",
    "Create documentation website for R packages",
    "Extension of base R plot functions",
    "Extract from model objects",
    "Extract information from model objects"
  ),
  check.names = FALSE
)
```

```r
tt(dat) |> format_tt(j = 1, markdown = TRUE)
```

Table 4: Vincent sometimes contributes to these R packages.

Package (link)	Purpose
marginaleffects	Interpreting statistical models
modelsummary	Data and model summaries
tinytable	Draw beautiful tables easily
countrycode	Convert country codes and names
WDI	Download data from the World Bank
softbib	Software bibliographies in R
tinysnapshot	Snapshots for unit tests using ‘tinytest‘
altdoc	Create documentation website for R packages
plot2	Extension of base R plot functions
parameters	Extract from model objects
insight	Extract information from model objects

2 Style

The main styling function for the `tinytable` package is `style_tt()`. Via this function, you can access three main interfaces to customize tables:

1. A general interface to frequently used style choices which works for both HTML and LaTeX (PDF): colors, font style and size, row and column spans, etc. This is accessed through several distinct arguments in the `style_tt()` function, such as `italic`, `color`, etc.
2. A specialized interface which allows users to use the powerful `tabularray` package to customize LaTeX tables. This is accessed by passing `tabularray` settings as strings to the `tabularray_inner` and `tabularray_outer` arguments of `style_tt()`.
3. A specialized interface which allows users to use the powerful `Bootstrap` framework to customize HTML tables. This is accessed by passing CSS declarations and rules to the `bootstrap_css` and `bootstrap_css_rule` arguments of `style_tt()`.

These functions can be used to customize rows, columns, or individual cells. They control many features, including:

- Text color

- Background color
- Widths
- Heights
- Alignment
- Text Wrapping
- Column and Row Spacing
- Cell Merging
- Multi-row or column spans
- Border Styling
- Font Styling: size, underline, italic, bold, strikethrough, etc.
- Header Customization

The `style_*()` functions can modify individual cells, or entire columns and rows. The portion of the table that is styled is determined by the `i` (rows) and `j` (columns) arguments.

2.1 Cells, rows, columns

To style individual cells, we use the `style_cell()` function. The first two arguments—`i` and `j`—identify the cells of interest, by row and column numbers respectively. To style a cell in the 2nd row and 3rd column, we can do:

```
tt(x) |>
  style_tt(
    i = 2,
    j = 3,
    background = "black",
    color = "white")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

The `i` and `j` accept vectors of integers to modify several cells at once:

```
tt(x) |>
  style_tt(
    i = 2:3,
```

```
    j = c(1, 3, 4),
    italic = TRUE,
    color = "orange")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	*160*	*110*	3.9
22.8	4	*108*	*93*	3.85
21.4	6	258	110	3.08

We can style all cells in a table by omitting both the `i` and `j` arguments:

```
tt(x) |> style_tt(color = "orange")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

We can style entire rows by omitting the `j` argument:

```
tt(x) |> style_tt(i = 1:2, color = "orange")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

We can style entire columns by omitting the `i` argument:

```r
tt(x) |> style_tt(j = c(2, 4), bold = TRUE)
```

mpg	**cyl**	disp	**hp**	drat
21	**6**	160	**110**	3.9
21	**6**	160	**110**	3.9
22.8	**4**	108	**93**	3.85
21.4	**6**	258	**110**	3.08

The `j` argument accepts integer vectors, but also a string with a Perl-style regular expression, which makes it easier to select columns by name:

```r
tt(x) |> style_tt(j = "mpg|drat", color = "orange")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

Of course, we can also call the `style_tt()` function several times to apply different styles to different parts of the table:

```r
tt(x) |>
  style_tt(i = 1, j = 1:2, color = "orange") |>
  style_tt(i = 1, j = 3:4, color = "green")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

2.2 Colors

The `color` and `background` arguments in the `style_tt()` function are used for specifying the text color and the background color for cells of a table created by the `tt()` function. This argument plays a crucial role in enhancing the visual appeal and readability of the table, whether it's rendered in LaTeX or HTML format. The way we specify colors differs slightly between the two formats:

For HTML Output:

- Hex Codes: You can specify colors using hexadecimal codes, which consist of a `#` followed by 6 characters (e.g., `#CC79A7`). This allows for a wide range of colors.
- Keywords: There's also the option to use color keywords for convenience. The supported keywords are basic color names like `black`, `red`, `blue`, etc.

For LaTeX Output:

- Hexadecimal Codes: Similar to HTML, you can use hexadecimal codes. However, in LaTeX, you need to include these codes as strings (e.g., `"#CC79A7"`).
- Keywords: LaTeX supports a different set of color keywords, which include standard colors like `black`, `red`, `blue`, as well as additional ones like `cyan`, `darkgray`, `lightgray`, etc.
- Color Blending: An advanced feature in LaTeX is color blending, which can be achieved using the `xcolor` package. You can blend colors by specifying ratios (e.g., `white!80!blue` or `green!20!red`).
- Luminance Levels: The `ninecolors` package in LaTeX offers colors with predefined luminance levels, allowing for more nuanced color choices (e.g., "azure4", "magenta8").

Note that the keywords used in LaTeX and HTML are slightly different.

```
tt(x) |> style_tt(i = 1:4, j = 1, color = "#FF5733")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

Note that when using Hex codes in a LaTeX table, we need extra declarations in the LaTeX preamble. See `?tt` for details.

2.3 Fonts

The font size is specified in em units.

```
tt(x) |> style_tt(j = "mpg|hp|qsec", fontsize = 1.5)
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

2.4 Spanning cells

Sometimes, it can be useful to make a cell stretch across multiple colums, for example when we want to insert a label. To achieve this, we can use the `colspan` argument. Here, we make the 2nd cell of the 2nd row stretch across three columns:

```
tt(x)|> style_tt(
  i = 2, j = 2,
  colspan = 3,
  align = "c",
  color = "white",
  background = "black")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21		6		3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

Here is the original table for comparison:

```
tt(x)
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

2.5 Headers

The header can be omitted from the table by deleting the column names in the `x` data frame:

```
k <- x
colnames(k) <- NULL
tt(k)
```

21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

The first is row 0, and higher level headers (ex: column spanning labels) have negative indices like -1. They can be styled as expected:

```
tt(x) |> style_tt(i = 0, color = "white", background = "black")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

When styling columns without specifying `i`, the headers are styled in accordance with the rest of the column:

```r
tt(x) |> style_tt(j = 2:3, color = "white", background = "black")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

2.6 Conditional styling

We can use the standard `which` function from Base `R` to create indices and apply conditional stying on rows. And we can use a regular expression in `j` to apply conditional styling on columns:

```r
k <- mtcars[1:10, c("mpg", "am", "vs")]

tt(k) |>
  style_tt(
    i = which(k$am == k$vs),
    background = "teal",
    color = "white")
```

mpg	am	vs
21	1	0
21	1	0
22.8	1	1
21.4	0	1
18.7	0	0
18.1	0	1
14.3	0	0
24.4	0	1
22.8	0	1
19.2	0	1

2.7 Vectorized styling (heatmaps)

The `color`, `background`, and `fontsize` arguments are vectorized. This allows easy specification of different colors in a single call:

```
tt(x) |>
  style_tt(
    i = 1:4,
    color = c("red", "blue", "green", "orange"))
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

When using a single value for a vectorized argument, it gets applied to all values:

```
tt(x) |>
  style_tt(
    j = 2:3,
    color = c("orange", "green"),
    background = "black")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

We can also produce more complex heatmap-like tables to illustrate different font sizes in em units:

```
# font sizes
fs <- seq(.1, 2, length.out = 20)

# headless table
```

```r
k <- data.frame(matrix(fs, ncol = 5))
colnames(k) <- NULL

# colors
bg <- hcl.colors(20, "Inferno")
fg <- ifelse(as.matrix(k) < 1.7, tail(bg, 1), head(bg, 1))

# table
tt(k, width = .7, theme = "void") |>
  style_tt(j = 1:5, align = "ccccc") |>
  style_tt(
    i = 1:4,
    j = 1:5,
    color = fg,
    background = bg,
    fontsize = fs)
```

..	0.5	0.9	1.3	1.7
0.2	0.6	1	1.4	1.8
0.3	0.7	1.1	1.5	1.9
0.4	0.8	1.2	1.6	2

2.8 Lines (borders)

The `style_tt` function allows us to customize the borders that surround eacell of a table, as well horizontal and vertical rules. To control these lines, we use the `line`, `line_width`, and `line_color` arguments. Here's a brief overview of each of these arguments:

- `line`: This argument specifies where solid lines should be drawn. It is a string that can consist of the following characters:
  - `"t"`: Draw a line at the top of the cell, row, or column.
  - `"b"`: Draw a line at the bottom of the cell, row, or column.
  - `"l"`: Draw a line at the left side of the cell, row, or column.
  - `"r"`: Draw a line at the right side of the cell, row, or column.
  - You can combine these characters to draw lines on multiple sides, such as `"tbl"` to draw lines at the top, bottom, and left sides of a cell.
- `line_width`: This argument controls the width of the solid lines in em units (default: 0.1 em). You can adjust this value to make the lines thicker or thinner.

- `line_color`: Specifies the color of the solid lines. You can use color names, hexadecimal codes, or other color specifications to define the line color.

Here is an example where we draw lines around every border ("t", "b", "l", and "r") of specified cells.

```
tt(x, theme = "void") |>
    style_tt(
        i = 0:3,
        j = 1:3,
        line = "tblr",
        line_width = 0.4,
        line_color = "orange")
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

And here is an example with horizontal rules:

```
tt(x, theme = "void") |>
    style_tt(i = 0, line = "t", line_color = "orange", line_width = 0.4) |>
    style_tt(i = 0, line = "b", line_color = "purple", line_width = 0.2) |>
    style_tt(i = 4, line = "b", line_color = "orange", line_width = 0.4)
```

mpg	cyl	disp	hp	drat
21	6	160	110	3.9
21	6	160	110	3.9
22.8	4	108	93	3.85
21.4	6	258	110	3.08

3 Tiny plots and images

The `plot_tt()` function can embed images and plots in a `tinytable`. We can insert images by specifying their paths and positions (`i`/`j`).

3.1 Inserting images in tables

To insert images in a table, we use the `plot_tt()` function. The `path_img` values must be relative to the main document saved by `save_tt()` or to the Quarto (or Rmarkdown) document in which the code is executed.

```
dat <- data.frame(
  Species = c("Spider", "Squirrel"),
  Image = ""
)

img <- c(
  "../man/figures/spider.png",
  "../man/figures/squirrel.png"
)

tt(dat) |>
  plot_tt(j = 2, images = img, height = 3)
```

Species	Image
Spider	
Squirrel	

In HTML tables, it is possible to insert tables directly from a web address, but not in LaTeX.

3.2 Inline plots

We can draw inline plots three ways, with

1. Built-in templates for histograms, density plots, and bar plots

2. Custom plots using base `R` plots.
3. Custom plots using `ggplot2`.

To draw custom plots, one simply has to define a custom function, whose structure we illustrate below.

3.2.1 Built-in plots

There are several types of inline plots available by default. For example,

```
plot_data <- list(mtcars$mpg, mtcars$hp, mtcars$qsec)

dat <- data.frame(
  Variables = c("mpg", "hp", "qsec"),
  Histogram = "",
  Density = "",
  Bar = ""
)

tt(dat) |>
  plot_tt(j = 2, fun = "histogram", data = plot_data) |>
  plot_tt(j = 3, fun = "density", data = plot_data, color = "darkgreen") |>
  plot_tt(j = 4, fun = "bar", data = list(2, 3, 6), color = "orange") |>
  style_tt(j = 2:4, align = "c")
```

Variables	Histogram	Density	Bar
mpg			
hp			
qsec			

3.2.2 Custom plots: Base `R`

Important: Custom functions must have `...` as an argument.

To create a custom inline plot using Base `R` plotting functions, we create a function that returns another function. `tinytable` will then call that second function internally to generate the plot.

This is easier than it sounds! For example:

```r
f <- function(d, ...) {
  function() hist(d, axes = FALSE, ann = FALSE, col = "lightblue")
}

plot_data <- list(mtcars$mpg, mtcars$hp, mtcars$qsec)

dat <- data.frame(Variables = c("mpg", "hp", "qsec"), Histogram = "")

tt(dat) |>
  plot_tt(j = 2, fun = f, data = plot_data)
```

Variables	Histogram
mpg	
hp	
qsec	

3.2.3 Custom plots: `ggplot2`

Important: Custom functions must have `...` as an argument.

To create a custom inline plot using `ggplot2`, we create a function that returns a `ggplot` object:

```r
library(ggplot2)

f <- function(d, color = "black", ...) {
  d <- data.frame(x = d)
  ggplot(d, aes(x = x)) +
    geom_histogram(bins = 30, color = color, fill = color) +
    scale_x_continuous(expand=c(0,0)) +
    scale_y_continuous(expand=c(0,0)) +
    theme_void()
}

plot_data <- list(mtcars$mpg, mtcars$hp, mtcars$qsec)

tt(dat) |>
  plot_tt(j = 2, fun = f, data = plot_data, color = "pink")
```

Variables	Histogram
mpg	
hp	
qsec	

We can insert arbitrarily complex plots by customizing the `ggplot2` call:

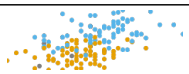
```
library(palmerpenguins)

# split data by species
dat <- split(penguins, penguins$species)
body <- lapply(dat, \(x) x$body_mass_g)
flip <- lapply(dat, \(x) x$flipper_length_mm)

# create nearly empty table
tab <- data.frame(
  "Species" = names(dat),
  "Body Mass" = "",
  "Flipper Length" = "",
  "Body vs. Flipper" = "",
  check.names = FALSE
)

# custom ggplot2 function to create inline plot
f <- function(d, ...) {
  ggplot(d, aes(x = flipper_length_mm, y = body_mass_g, color = sex)) +
    geom_point(size = .2) +
    scale_x_continuous(expand=c(0,0)) +
    scale_y_continuous(expand=c(0,0)) +
    scale_color_manual(values = c("#E69F00", "#56B4E9")) +
    theme_void() +
    theme(legend.position = "none")
}

# `tinytable` calls
tt(tab) |>
  plot_tt(j = 2, fun = "histogram", data = body, height = 2) |>
  plot_tt(j = 3, fun = "density", data = flip, height = 2) |>
  plot_tt(j = 4, fun = f, data = dat, height = 2) |>
  style_tt(j = 2:4, align = "c")
```

Species	Body Mass	Flipper Length	Body vs. Flipper
Adelie			
Chinstrap			
Gentoo			

4 Groups and labels

The `group_tt()` function can label groups of rows (`i`) or columns (`j`).

4.1 Rows

The `i` argument accepts a named list of integers. The numbers identify the positions where row group labels are to be inserted. The names includes the text that should be inserted:

```
dat <- mtcars[1:9, 1:8]

tt(dat) |>
  group_tt(i = list(
    "I like (fake) hamburgers" = 3,
    "She prefers halloumi" = 4,
    "They love tofu" = 7))
```

mpg	cyl	disp	hp	drat	wt	qsec	vs
21	6	160	110	3.9	2.62	16.46	0
21	6	160	110	3.9	2.875	17.02	0
I like (fake) hamburgers							
22.8	4	108	93	3.85	2.32	18.61	1
She prefers halloumi							
21.4	6	258	110	3.08	3.215	19.44	1
18.7	8	360	175	3.15	3.44	17.02	0
18.1	6	225	105	2.76	3.46	20.22	1
They love tofu							
14.3	8	360	245	3.21	3.57	15.84	0
24.4	4	146.7	62	3.69	3.19	20	1
22.8	4	140.8	95	3.92	3.15	22.9	1

We can style group rows in the same way as regular rows:

```
tt(dat) |>
  group_tt(
    i = list(
      "I like (fake) hamburgers" = 3,
      "She prefers halloumi" = 4,
      "They love tofu" = 7)) |>
  style_tt(
    i = c(3, 5, 9),
    align = "c",
    color = "white",
    background = "gray",
    bold = TRUE)
```

mpg	cyl	disp	hp	drat	wt	qsec	vs
21	6	160	110	3.9	2.62	16.46	0
21	6	160	110	3.9	2.875	17.02	0
I like (fake) hamburgers							
22.8	4	108	93	3.85	2.32	18.61	1
She prefers halloumi							
21.4	6	258	110	3.08	3.215	19.44	1
18.7	8	360	175	3.15	3.44	17.02	0
18.1	6	225	105	2.76	3.46	20.22	1
They love tofu							
14.3	8	360	245	3.21	3.57	15.84	0
24.4	4	146.7	62	3.69	3.19	20	1
22.8	4	140.8	95	3.92	3.15	22.9	1

4.2 Columns

The syntax for column groups is very similar, but we use the `j` argument instead. The named list specifies the labels to appear in column-spanning labels, and the values must be a vector of consecutive and non-overlapping integers that indicate which columns are associated to which labels:

```
tt(dat) |>
  group_tt(
    j = list(
      "Hamburgers" = 1:3,
      "Halloumi" = 4:5,
      "Tofu" = 7))
```

Hamburgers			Halloumi			Tofu	
mpg	cyl	disp	hp	drat	wt	qsec	vs
21	6	160	110	3.9	2.62	16.46	0
21	6	160	110	3.9	2.875	17.02	0
22.8	4	108	93	3.85	2.32	18.61	1
21.4	6	258	110	3.08	3.215	19.44	1
18.7	8	360	175	3.15	3.44	17.02	0
18.1	6	225	105	2.76	3.46	20.22	1
14.3	8	360	245	3.21	3.57	15.84	0
24.4	4	146.7	62	3.69	3.19	20	1
22.8	4	140.8	95	3.92	3.15	22.9	1

Here is a table with both row and column headers, as well as some styling:

```r
dat <- mtcars[1:9, 1:8]
tt(dat) |>
  group_tt(
    i = list("I like (fake) hamburgers" = 3,
             "She prefers halloumi" = 4,
             "They love tofu" = 7),
    j = list("Hamburgers" = 1:3,
             "Halloumi" = 4:5,
             "Tofu" = 7)) |>
  style_tt(
    i = c(3, 5, 9),
    align = "c",
    background = "teal",
    color = "white") |>
  style_tt(i = -1, color = "teal")
```

Hamburgers			Halloumi			Tofu	
mpg	cyl	disp	hp	drat	wt	qsec	vs
21	6	160	110	3.9	2.62	16.46	0
21	6	160	110	3.9	2.875	17.02	0
I like (fake) hamburgers							
22.8	4	108	93	3.85	2.32	18.61	1
She prefers halloumi							
21.4	6	258	110	3.08	3.215	19.44	1
18.7	8	360	175	3.15	3.44	17.02	0
18.1	6	225	105	2.76	3.46	20.22	1
They love tofu							
14.3	8	360	245	3.21	3.57	15.84	0
24.4	4	146.7	62	3.69	3.19	20	1
22.8	4	140.8	95	3.92	3.15	22.9	1

5 HTML customization

The HTML customization options described in this section are not available for LaTeX (or PDF) documents. Please refer to the web documentation to read this part of the tutorial.

5.1 Themes

5.2 CSS declarations

5.3 CSS rules

6 LaTeX / PDF customization

6.1 Preamble

Warning: Some of the features of this package may require a recent version of the `tabularray` package. Please update your local LaTeX distribution before using `tinytable`.

In Rmarkdown and Quarto documents, `tinytable` will automatically populate your LaTeX preamble with the necessary packages and commands. When creating your own LaTeX documents, you should insert these commands in the preamble:

```latex
\usepackage{tabularray}
\usepackage{float}
\usepackage{codehigh}
\usepackage{graphicx}
\usepackage[normalem]{ulem}
\UseTblrLibrary{booktabs}
\NewTableCommand{\tinytableDefineColor}[3]{\definecolor{#1}{#2}{#3}}
\newcommand{\tinytableTabularrayUnderline}[1]{\underline{#1}}
\newcommand{\tinytableTabularrayStrikeout}[1]{\sout{#1}}
```

6.2 Introduction to `tabularray`

`tabularray` offers a robust solution for creating and managing tables in LaTeX, standing out for its flexibility and ease of use. It excels in handling complex table layouts and offers enhanced functionality compared to traditional LaTeX table environments. This package is particularly useful for users requiring advanced table features, such as complex cell formatting, color management, and versatile table structures.

A key feature of Tabularray is its separation of style from content. This approach allows users to define the look and feel of their tables (such as color, borders, and text alignment) independently from the actual data within the table. This separation simplifies the process of formatting tables and enhances the clarity and maintainability of LaTeX code. The `tabularray` documentation is fantastic. It will teach you how to customize virtually every aspect of your tables: https://ctan.org/pkg/tabularray?lang=en

Tabularray introduces a streamlined interface for specifying table settings. It employs two types of settings blocks: Inner and Outer. The Outer block is used for settings that apply to the entire table, like overall alignment, while the Inner block handles settings for specific elements like columns, rows, and cells. The `style_tt()` function includes `tabularray_inner` and `tabularray_outer` arguments to set these respective features.

Consider this `tabularray` example, which illustrates the use of inner settings:

```latex
\begin{table}
\centering
\begin{tblr}[         %% tabularray outer open
]                     %% tabularray outer close
{                     %% tabularray inner open
column{1-4}={halign=c},
hlines = {bg=white},
vlines = {bg=white},
cell{1,6}{odd} = {bg=teal7},
```

```
cell{1,6}{even} = {bg=green7},
cell{2,4}{1,4} = {bg=red7},
cell{3,5}{1,4} = {bg=purple7},
cell{2}{2} = {r=4,c=2}{bg=azure7},
}                       %% tabularray inner close
mpg & cyl & disp & hp \\
21 & 6 & 160 & 110 \\
21 & 6 & 160 & 110 \\
22.8 & 4 & 108 & 93 \\
21.4 & 6 & 258 & 110 \\
18.7 & 8 & 360 & 175 \\
\end{tblr}
\end{table}
```

The Inner block, enclosed in `{}`, defines specific styles like column formats (`column{1-4}={halign=c}`), horizontal and vertical line colors (`hlines={fg=white}`, `vlines={fg=white}`), and cell colorations (`cell{1,6}{odd}={bg=teal7}`, etc.). The last line of the inner block also species that the second cell of row 2 (`cell{2}{2}`) should span 4 rows and 2 columns (`{r=4,c=3}`), be centered (`halign=c`), and with a background color with the 7th luminance level of the azure color (`bg=azure7`).

We can create this code easily by passing a string to the `tabularray_inner` argument of the `style_tt()` function:

```
inner <- "
column{1-4}={halign=c},
hlines = {fg=white},
vlines = {fg=white},
cell{1,6}{odd} = {bg=teal7},
cell{1,6}{even} = {bg=green7},
cell{2,4}{1,4} = {bg=red7},
cell{3,5}{1,4} = {bg=purple7},
cell{2}{2} = {r=4,c=2}{bg=azure7},
"
mtcars[1:5, 1:4] |>
  tt(theme = "void") |>
  style_tt(tabularray_inner = inner)
```

Table 5: LaTeX table with colors and a spanning cell.

mpg	cyl	disp	hp
21	6		110
21			110
22.8			93
21.4			110
18.7	8	360	175

6.3 `tabularray` keys

Inner specifications:

Key	Description and Values	Initial Value
`rulesep`	space between two hlines or vlines	`2pt`
`stretch`	stretch ratio for struts added to cell text	`1`
`abovesep`	set vertical space above every row	`2pt`
`belowsep`	set vertical space below every row	`2pt`
`rowsep`	set vertical space above and below every row	`2pt`
`leftsep`	set horizontal space to the left of every column	`6pt`
`rightsep`	set horizontal space to the right of every column	`6pt`
`colsep`	set horizontal space to both sides of every column	`6pt`
`hspan`	horizontal span algorithm: `default`, `even`, or `minimal`	`default`
`vspan`	vertical span algorithm: `default` or `even`	`default`
`baseline`	set the baseline of the table	`m`

Outer specifications:

Key	Description and Values	Initial Value
`baseline`	set the baseline of the table	`m`
`long`	change the table to a long table	None
`tall`	change the table to a tall table	None
`expand`	you need this key to use verb commands	None

Cells:

Key	Description and Values	Initial Value
`halign`	horizontal alignment: `l` (left), `c` (center), `r` (right) or `j` (justify)	`j`
`valign`	vertical alignment: `t` (top), `m` (middle), `b` (bottom), `h` (head) or `f` (foot)	`t`
`wd`	width dimension	None
`bg`	background color name	None
`fg`	foreground color name	None
`font`	font commands	None
`mode`	set cell mode: `math`, `imath`, `dmath` or `text`	None
`cmd`	execute command for the cell text	None
`preto`	prepend text to the cell	None
`appto`	append text to the cell	None
`r`	number of rows the cell spans	1
`c`	number of columns the cell spans	1

Rows:

Key	Description and Values	Initial Value
`halign`	horizontal alignment: `l` (left), `c` (center), `r` (right) or `j` (justify)	`j`
`valign`	vertical alignment: `t` (top), `m` (middle), `b` (bottom), `h` (head) or `f` (foot)	`t`
`ht`	height dimension	None
`bg`	background color name	None
`fg`	foreground color name	None
`font`	font commands	None
`mode`	set mode for row cells: `math`, `imath`, `dmath` or `text`	None
`cmd`	execute command for every cell text	None
`abovesep`	set vertical space above the row	`2pt`
`belowsep`	set vertical space below the row	`2pt`
`rowsep`	set vertical space above and below the row	`2pt`
`preto`	prepend text to every cell (like `>` specifier in `rowspec`)	None
`appto`	append text to every cell (like `<` specifier in `rowspec`)	None

Columns:

Key	Description and Values	Initial Value
`halign`	horizontal alignment: `l` (left), `c` (center), `r` (right) or `j` (justify)	`j`

Key	Description and Values	Initial Value
`valign`	vertical alignment: `t` (top), `m` (middle), `b` (bottom), `h` (head) or `f` (foot)	`t`
`wd`	width dimension	None
`co`	coefficient for the extendable column (`X` column)	None
`bg`	background color name	None
`fg`	foreground color name	None
`font`	font commands	None
`mode`	set mode for column cells: `math`, `imath`, `dmath` or `text`	None
`cmd`	execute command for every cell text	None
`leftsep`	set horizontal space to the left of the column	`6pt`
`rightsep`	set horizontal space to the right of the column	`6pt`
`colsep`	set horizontal space to both sides of the column	`6pt`
`preto`	prepend text to every cell (like `>` specifier in `colspec`)	None
`appto`	append text to every cell (like `<` specifier in `colspec`)	None

hlines:

Key	Description and Values	Initial Value
`dash`	dash style: `solid`, `dashed` or `dotted`	`solid`
`text`	replace hline with text (like `!` specifier in `rowspec`)	None
`wd`	rule width dimension	`0.4pt`
`fg`	rule color name	None
`leftpos`	crossing or trimming position at the left side	`1`
`rightpos`	crossing or trimming position at the right side	`1`
`endpos`	adjust leftpos/rightpos for only the leftmost/rightmost column	`false`

vlines:

Key	Description and Values	Initial Value
`dash`	dash style: `solid`, `dashed` or `dotted`	`solid`
`text`	replace vline with text (like `!` specifier in `colspec`)	None
`wd`	rule width dimension	`0.4pt`
`fg`	rule color name	None
`abovepos`	crossing or trimming position at the above side	`0`
`belowpos`	crossing or trimming position at the below side	`0`